

Conteúdo 10:

Classes Abstratas e Polimorfismo

Professora: **Me. Alana Neo**

Slides baseados no material do Prof. Me. Douglas Thames
de Araujo

Classes Abstratas

- Pode-se dizer que as **classes abstratas** servem como “**modelo**” para outras classes que dela **herdem**, **não** podendo ser **instanciada** por si só.
- Para ter um objeto de uma classe abstrata é necessário criar uma classe mais especializada herdando dela e então instanciar essa nova classe. Os métodos da classe abstrata devem então serem sobrescritos nas classes filhas.
- Por exemplo, é definido que a classe “Animal” seja herdada pelas subclasses “Gato”, “Cachorro”, “Cavalo”, mas ela mesma nunca pode ser instanciada.

Classes Abstratas

```
Conta.java x
Código-Fonte  Histórico
13  abstract class Conta {
14      private double saldo;
15
16      public void setSaldo(double saldo) {
17          this.saldo = saldo;
18      }
19
20      public double getSaldo() {
21          return saldo;
22      }
23
24      public abstract void imprimeExtrato();
25
26  }
```

Classes Abstratas

```
ContaPoupanca.java x
Código-Fonte  Histórico
15 public class ContaPoupanca extends Conta {
16
17     @Override
18     public void imprimeExtrato() {
19         System.out.println("### Extrato da Conta ###");
20
21         SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy HH:mm:ss");
22         Date date = new Date();
23
24         System.out.println("Saldo: " + this.getSaldo());
25         System.out.println("Data: " + sdf.format(date));
26
27     }
28
29 }
```

Classes Abstratas

- No exemplo da **classe ContaPoupanca**, o método “`imprimeExtrato()`” tem uma *annotation* conhecida como **@Override**, significando que estamos sobrescrevendo o método da superclasse.
- Na classe abstrata “Conta” os **métodos** que são abstratos têm um comportamento diferente, por isso não possuem corpo. Ou seja, **as subclasses que estão herdando precisam desse método** mas não de forma genérica, onde permite inserir as particularidades de cada subclasse.

Classes Abstratas

```
TesteConta.java x
Código-Fonte  Histórico  [Icons]

12  public class TesteConta {
13
14      /**
15       * @param args the command line arguments
16       */
17      public static void main(String[] args) {
18          Conta cp = new ContaPoupanca();
19
20          cp.setSaldo(2121);
21          cp.imprimeExtrato();
22      }
23
24  }
```

Polimorfismo

- O polimorfismo é mais um princípio fundamental da orientação a objetos;
- Significa, ao pé da letra, “várias formas”;
- **Não confundir com sobrecarga (*overload*)**
- Ele permite que classes pertencentes a uma mesma linha de herança possuam comportamentos diferentes para o mesmo Método.

Polimorfismo X Sobrecarga

- **Sobrecarga de métodos** (overload) é um conceito do polimorfismo que consiste basicamente em criar variações de um mesmo método, ou seja, a criação de dois ou mais métodos com **nomes** totalmente **iguais em uma classe**.
- A **Sobrecarga** permite que utilizemos o mesmo nome em mais de um método contanto que suas listas de argumentos sejam diferentes para que seja feita a separação dos mesmos.

Polimorfismo

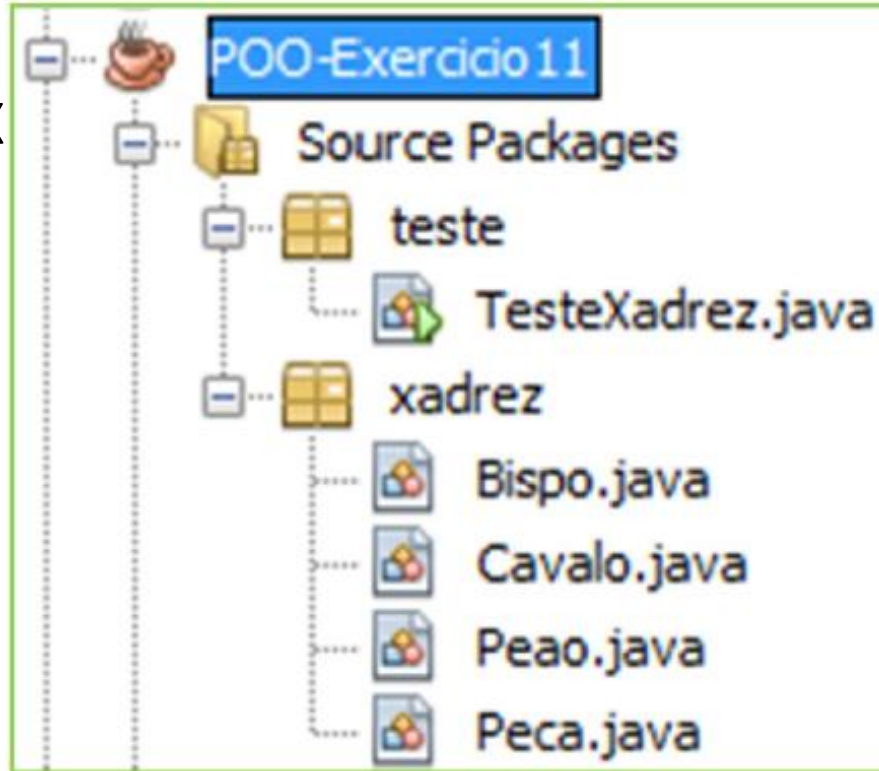
- **Exemplo simples:** em um jogo de xadrez temos várias peças, cada peça se movimenta, porém cada tipo de peça se movimenta de uma maneira diferente (o peão vai pra frente, o cavalo anda em L, o bispo em diagonal e assim por diante)
- Desta maneira, **todos** (peão, cavalo e bispo) **são peças**, porém cada um se movimenta de uma maneira diferente!

Polimorfismo

- O polimorfismo em Java se manifesta apenas na chamada de métodos.
- Então ao passar uma mensagem para um objeto peça, dizendo para ele se mover, o Java identificará que tipo de peça é e fará o movimento de acordo com o tipo.
- Essa decisão é tomada em tempo de execução (*runtime*) pelo Java.

Polimorfismo – Exemplo Xadrez

- Exemplo do X



Polimorfismo – Exemplo Xadrez

```
1 package xadrez;  
2  
3 public abstract class Peca {  
4     public abstract void mover();  
5 }
```

```
3 public class Peao extends Peca{  
4     @Override  
5     public void mover() {  
6         System.out.println("ANDAR PRA FRENTE");  
7     }  
8 }
```

```
3 public class Cavalo extends Peca{  
4     @Override  
5     public void mover() {  
6         System.out.println("ANDAR EM 'L'");  
7     }  
8 }
```

```
3 public class Bispo extends Peca{  
4     @Override  
5     public void mover() {  
6         System.out.println("ANDAR NA DIAGONAL");  
7     }  
8 }
```

Polimorfismo – Exemplo Xadrez

```
8 public class TesteXadrez {
9     public static void main(String[] args) {
10         Peca peao = new Peao();
11         Peca cavalo = new Cavalo();
12         Peca bispo = new Bispo();
13
14         peao.mover();
15         cavalo.mover();
16         bispo.mover();
17     }
18 }
```

Apache Tomcat 8.0.3.0

POO-Exercicio11 (run)

run:

ANDAR PRA FRENTE

ANDAR EM 'L'

ANDAR NA DIAGONAL

BUILD SUCCESSFUL (total time: 0 seconds)

Polimorfismo

- Observe que os três objetos (peao, cavalo e bispo) **são do tipo Peca**.
- Isso pode ser confirmado através do uso do *instance of* do Java:

```
System.out.println(peao instanceof Peca);  
System.out.println(cavalo instanceof Peca);  
System.out.println(bispo instanceof Peca);
```

Apache Tomcat 8.0.3.0 POO-Exercicio11 (run)

ANDAR NA DIAGONAL

true
true
true

BUILD SUCCESSFUL (total time: 0 seconds)

Vamos praticar?

- Crie uma estrutura de herança Animal e pelo menos 3 subclasses para demonstrar o polimorfismo utilizando classes de animais.
- Crie o método comunicar(), movimentar() e imprimirDados() na classe Animal. Os métodos devem imprimir uma mensagem de acordo com a classe que está realizando a chamada do mesmo.
- Nas classes descendentes de animais, sobrescreva os métodos citados de acordo com os tipos de animais e insira pelo menos 4 atributos.
- Confirme que todos são tipos de Animal com o uso de instance of do Java. Insira novos dados nas classes criadas e execute os métodos de cada uma.



Dúvidas e Questionamentos



INSTITUTO FEDERAL
Mato Grosso do Sul

alana.neo@ifms.edu.br